

Deep Reinforcement Learning on Atari Games Pong and Breakout

Flavien Geoffray
flavien.geoffray@epita.fr

EPITA
Reinforcement Learning Project

November 26, 2025

1 Introduction

Reinforcement Learning (RL) has emerged as a powerful paradigm for training autonomous agents to make sequences of decisions in complex environments. Unlike supervised learning, where a model learns from labeled data, RL agents learn by interacting with an environment, receiving feedback in the form of rewards or penalties. The goal is to learn a policy that maximizes the cumulative reward over time. A significant breakthrough in this field was the development of the Deep Q-Network (DQN) by Mnih et al. (2015), which combined Q-learning with deep convolutional neural networks (CNNs). This architecture allowed agents to learn directly from high-dimensional sensory inputs, such as raw pixels, achieving human-level performance on the Atari 2600 games suite.

This project implements and evaluates a DQN agent on two classic Atari games, *Pong* and *Breakout*, to understand the practical challenges and architectural requirements of Deep RL. The primary goal is to build the core components of the DQN algorithm from scratch, including Experience Replay and Target Networks, and to assess their effectiveness across environments with varying visual and strategic complexities. Additionally, this study analyzes the impact of neural network depth on performance by comparing a 2-layer Convolutional Neural Network (CNN), the original DeepMind implementation, against a deeper 3-layer architecture. Through this comparative analysis, the project aims to demonstrate how architectural choices influence an agent’s ability to learn robust control policies from raw pixel data.

2 Implementation Details

2.1 Environment and Preprocessing

The agent interacts with the Atari environments via the `Gymnasium` library. Raw Atari frames are high-dimensional (210×160 RGB pixels) and difficult to process directly. To reduce computational complexity and stabilize training, I implemented a standard preprocessing pipeline in `dqn/env.py`:

- **Grayscale Conversion & Resizing:** Input frames are converted to grayscale and down-scaled to 84×84 pixels.
- **Normalization:** Pixel values are normalized from $[0, 255]$ to $[0, 1]$ (float32).
- **Frame Skipping:** The agent selects an action every 4^{th} frame. The selected action is repeated for the skipped frames, significantly speeding up training.
- **Frame Stacking:** The last 4 frames are stacked to form the state observation ($84 \times 84 \times 4$). This allows the network to perceive temporal dynamics like the ball’s velocity and direction.

2.2 Neural Network Architectures

Two distinct Convolutional Neural Network (CNN) architectures were implemented in `dqn/dqn.py` to approximate the Q-function.

2.2.1 Model A: DeepMind DQN (2 Layers)

A shallower network used as a baseline for less complex tasks or initial testing:

- **Input:** $84 \times 84 \times 4$ image stack.
- **Conv1:** 16 filters, kernel 8×8 , stride 4, ReLU.
- **Conv2:** 32 filters, kernel 4×4 , stride 2, ReLU.

- **FC1:** Fully Connected layer with 256 units.
- **Output:** Linear layer with $N_{actions}$ outputs.

2.2.2 Model B: Deep DQN (3 Layers)

A deeper architecture based on the Mnih et al. (2015) paper, used for more complex visual environments:

- **Conv1:** 32 filters, kernel 8×8 , stride 4, ReLU.
- **Conv2:** 64 filters, kernel 4×4 , stride 2, ReLU.
- **Conv3:** 64 filters, kernel 3×3 , stride 1, ReLU.
- **FC1:** Fully Connected layer with 512 units.
- **Output:** Linear layer with $N_{actions}$ outputs.

2.3 The DQN Agent

The agent logic is encapsulated in `dqn/agent.py` and `dqn/train.py`. Key components include:

- **Epsilon-Greedy Policy:** Linearly decays exploration (ϵ) from 1.0 to 0.1 over 1,000,000 steps to balance exploration and exploitation.
- **Experience Replay:** Transitions $(s, a, r, s', done)$ are stored in a cyclic buffer (implemented in `dqn/memory.py`). Training samples are drawn randomly to break the correlation between consecutive frames.
- **Target Network:** A frozen copy of the Q-network is used to compute target values. Its weights are updated periodically (every 10,000 steps) to prevent feedback loops and oscillation.
- **Optimization:** The Huber Loss (Smooth L1) is minimized using either RMSprop or Adam optimizers.

3 Evaluation: Pong

3.1 Game Description

Pong is a simulation of table tennis. The agent controls a paddle on the right side of the screen and competes against a computer-controlled opponent on the left.

- **Goal:** Pass the ball past the opponent's paddle.
- **Scoring:** +1 reward for scoring, -1 reward for missing the ball. The first to 21 points wins.
- **Actions:** 6 available actions (reduced effectively to UP, DOWN, and NOOP).

3.2 Training Configuration

The model was trained for 5,000 episodes using the Basic DQN (2-layer) architecture.

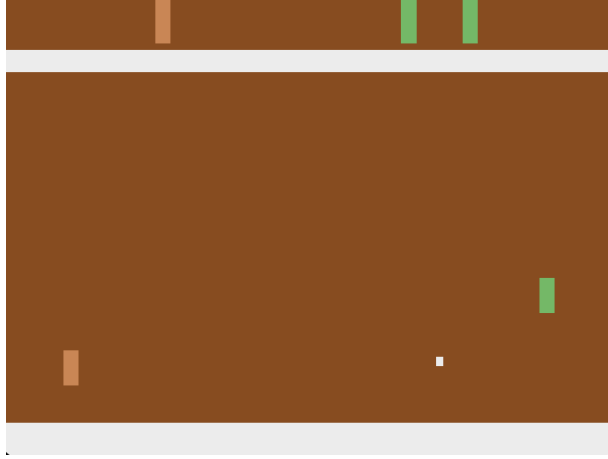


Figure 1: The Pong Environment (ALE/Pong-v5).

Parameter	Value
Environment	ALE/Pong-v5
Architecture	Basic DQN (2 Conv Layers)
Total Episodes	5,000
Memory Capacity	100,000
Batch Size	32
Learning Rate	2.5×10^{-4}
Target Update Freq.	10,000 steps
Gamma (γ)	0.99

Table 1: Hyperparameters for Pong

3.3 Training Results

The following plots visualize the agent’s learning progress.

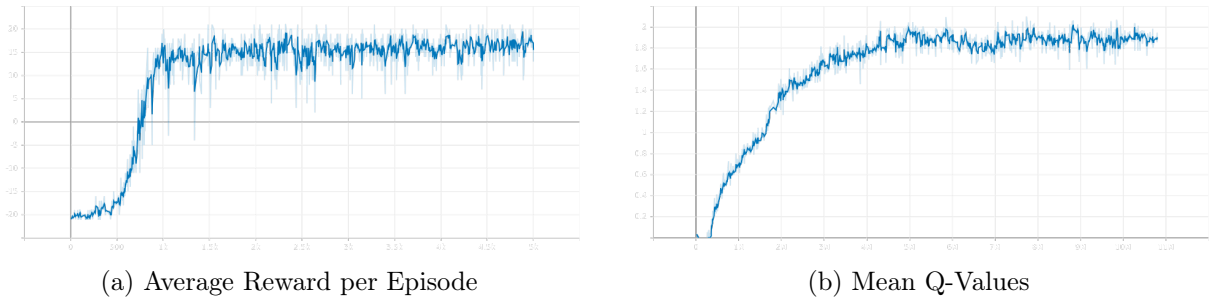


Figure 2: Training metrics for Pong.

3.4 Analysis

The agent successfully solved the environment. As shown in the reward graph, the agent starts with a score of -21 (losing every round) and steadily improves. By episode 1,300, the reward reaches roughly 12-13. By the end of training (5,000 episodes), the agent achieves an average reward of **19.5**, demonstrating a near-perfect level of play against the computer opponent. The stability of the Q-values indicates that the agent has learned a consistent value function.

4 Evaluation: Breakout

4.1 Game Description

Breakout requires the player to control a paddle at the bottom of the screen to bounce a ball upward and destroy layers of bricks.

- **Goal:** Destroy all bricks and keep the ball in play.
- **Scoring:** Points are awarded for destroying bricks. The ball moves faster as bricks are cleared.
- **Actions:** 4 actions (NOOP, FIRE, RIGHT, LEFT).
- **Difficulty:** This environment is significantly harder than Pong due to the precise timing required and the sparsity of rewards.



Figure 3: The Breakout Environment (ALE/Breakout-v5).

4.2 Training Configuration

Breakout was trained using the deeper 3-layer architecture (DQNv2) to handle the increased visual complexity. Note the increased memory capacity and learning start time compared to Pong.

Parameter	Value
Environment	ALE/Breakout-v5
Architecture	Deep DQN (3 Layers)
Total Episodes	10,000
Memory Capacity	1,000,000
Batch Size	32
Learning Rate	1.0×10^{-4} (Adam)
Target Update Freq.	10,000 steps
Learning Starts	50,000 steps

Table 2: Hyperparameters for Breakout

4.3 Training Results

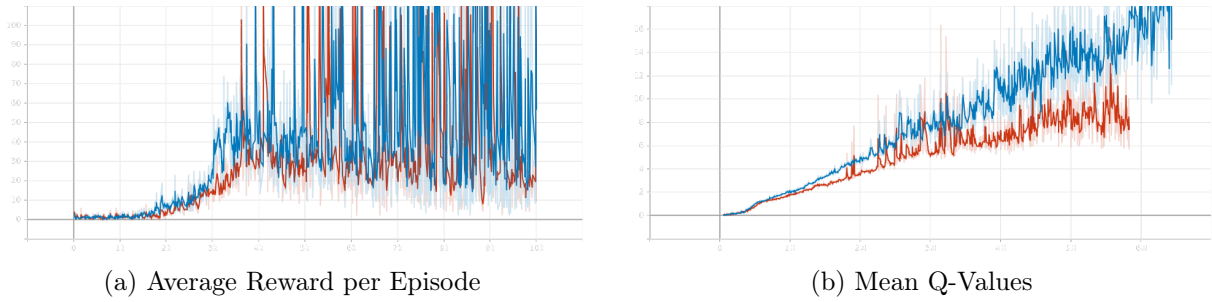


Figure 4: Training metrics for Breakout. The blue curve represents the Deep DQN (3-layer architecture), while the red curve corresponds to the DeepMind DQN (2-layer architecture).

4.4 Analysis

The agent shows promising learning in the Breakout environment, achieving an average reward of **70** by the end of 10,000 episodes. The blue curve in the plots represents the Deep DQN (3-layer architecture), while the red curve corresponds to the DeepMind DQN (2-layer architecture). While the 3-layer model demonstrates a good capability to score points, the 2-layer model, though slightly less performant, still proves effective at learning and scoring some points. This suggests that further training or architectural refinements could lead to higher scores and improved strategies for conserving lives, especially for the deeper network.

5 Conclusion

This project successfully demonstrated the effectiveness of Deep Q-Networks on Atari games. We showed that a standard DQN implementation can master Pong with relative ease, while a more complex game like Breakout requires a deeper network architecture and specific training strategies, such as large replay buffers.